

Here's a **complete, structured comparison of JUnit 5 vs Mockito**, along with **clear Java implementation examples** (including real-world usage).

1. What is JUnit 5 vs Mockito?

Feature	JUnit 5	Mockito
Type	Testing Framework	Mocking Framework
Purpose	Execute test cases	Create mock objects
Main Use	Unit Testing	Dependency Isolation
Package	org.junit.jupiter.*	org.mockito.*
Developed By	JUnit Team	Mockito Team
Works Alone?	✓ Yes	✗ Needs test framework (JUnit)

2. Similarities between JUnit 5 and Mockito

Similarity	Description
Used in Testing	Both are used for unit testing in Java
Integration	Mockito integrates with JUnit 5
Annotations Support	Both provide annotations
Improves Code Quality	Helps test logic and behavior
Open Source	Both are free and widely used

3. Differences (Core Concept)

Feature	JUnit 5	Mockito
Role	Test runner & assertions	Mocking dependencies
Focus	Test logic	Fake external objects
Key Use Case	Verify outputs	Control inputs/behavior

Execution	Runs test cases	Used inside test cases
Assertions	Yes (assertEquals)	No assertions
Mocking	✗ No	✓ Yes
Stubbing	✗ No	✓ Yes
Verification	✗ No	✓ Yes

□ 4. When to Use What?

Scenario	Use
Writing test cases	JUnit 5
Mocking database/API	Mockito
Testing business logic	Both
Avoiding real dependencies	Mockito

□ 5. Maven Dependencies

```

<dependencies>
  <!-- JUnit 5 -->
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.0</version>
    <scope>test</scope>
  </dependency>

  <!-- Mockito -->
  <dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <version>5.5.0</version>
    <scope>test</scope>
  </dependency>

  <!-- Mockito + JUnit 5 integration -->
  <dependency>
    <groupId>org.mockito</groupId>

```

```
<artifactId>mockito-junit-jupiter</artifactId>
<version>5.5.0</version>
<scope>test</scope>
</dependency>
</dependencies>
```

❑ 6. JUnit 5 Example (Basic Test)

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
```

```
class Calculator {
```

```
    int add(int a, int b) {
        return a + b;
    }
}
```

```
public class CalculatorTest {
```

```
    @Test
    void testAdd() {
        Calculator calc = new Calculator();
        int result = calc.add(5, 3);

        assertEquals(8, result);
    }
}
```

✓ **Focus:** Testing logic

❑ 7. Mockito Example (Mocking Dependency)

Scenario:

Service depends on Repository (DB)

Step 1: Classes

```
class UserRepository {
    String findUser() {
        return "Real User from DB";
    }
}
```

```
class UserService {  
  
    private UserRepository repo;  
  
    UserService(UserRepository repo) {  
        this.repo = repo;  
    }  
  
    String getUser() {  
        return repo.findUser();  
    }  
}
```

Step 2: Test with Mockito

```
import org.junit.jupiter.api.Test;  
import static org.mockito.Mockito.*;  
import static org.junit.jupiter.api.Assertions.*;  
  
class UserServiceTest {  
  
    @Test  
    void testGetUser() {  
  
        // Create mock  
        UserRepository mockRepo = mock(UserRepository.class);  
  
        // Define behavior  
        when(mockRepo.findUser()).thenReturn("Mock User");  
  
        // Inject mock  
        UserService service = new UserService(mockRepo);  
  
        // Call method  
        String result = service.getUser();  
  
        // Assertion  
        assertEquals("Mock User", result);  
  
        // Verify interaction  
        verify(mockRepo).findUser();  
    }  
}
```

✓ **Focus:** Mocking dependency

□ 8. JUnit 5 + Mockito (Best Practice)

```
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
```

```
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;
```

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {
```

```
    @Mock
    UserRepository repo;
```

```
    @InjectMocks
    UserService service;
```

```
    @Test
    void testGetUser() {
```

```
        when(repo.findUser()).thenReturn("Mock User");
```

```
        String result = service.getUser();
```

```
        assertEquals("Mock User", result);
        verify(repo).findUser();
```

```
    }
}
```

□ 9. Key Mockito Concepts

Concept	Example
Mock	mock(Class.class)
Stubbing	when().thenReturn()
Verify	verify()
InjectMocks	Auto inject dependencies
Spy	Partial mocking

❑ 10. Real-Time Example (Enterprise)

Layer	Tool Used
Controller Test	JUnit + Mockito
Service Test	JUnit + Mockito
Repository Test	JUnit (no mock)
External API	Mockito

❑ 11. Summary

Aspect	JUnit 5	Mockito
Purpose	Run tests	Mock dependencies
Works Together	✓ Yes	
Replacement	✗ Not replaceable	
Essential	✓	✓

✓ Final Understanding

- **JUnit 5 = “Test Runner + Assertions”**
- **Mockito = “Fake Objects + Behavior Control”**
- ❑ **Both are used together in real projects**

If you want next level ❑, I can give:

- ❑ 50+ Interview Questions (JUnit + Mockito)
- ❑ Spring Boot testing (Controller + Service + Repository)
- ❑ Advanced Mockito (Spy, ArgumentCaptor, doThrow)

Here's a **complete, real-world guide to Spring Boot testing** covering:

- ✓ Controller Layer
 - ✓ Service Layer
 - ✓ Repository Layer
 - ✓ Full working Java code examples
 - ✓ Best practices used in companies
-

□ 1. Spring Boot Testing Architecture

Controller → Service → Repository → Database

Layer	Testing Type	Tool
Controller	Web Layer Test	MockMvc + Mockito
Service	Unit Test	JUnit + Mockito
Repository	Integration Test	JPA + H2 DB

□ 2. Maven Dependencies

<dependencies>

```
<!-- Spring Boot Test -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

```
<!-- H2 Database for testing -->
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>test</scope>
</dependency>
```

</dependencies>

❑ 3. Example Application (Simple User API)

📄 Entity

```
import jakarta.persistence.*;

@Entity
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    // getters & setters
}
```

📄 Repository

```
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Long> {
}
```

📄 Service

```
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class UserService {

    private final UserRepository repo;

    public UserService(UserRepository repo) {
        this.repo = repo;
    }

    public List<User> getAllUsers() {
        return repo.findAll();
    }

    public User saveUser(User user) {
        return repo.save(user);
    }
}
```

❏ Controller

```
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService service;

    public UserController(UserService service) {
        this.service = service;
    }

    @GetMapping
    public List<User> getUsers() {
        return service.getAllUsers();
    }

    @PostMapping
    public User createUser(@RequestBody User user) {
        return service.saveUser(user);
    }
}
```

❏ 4. Service Layer Testing (Unit Test)

- ❏ Focus: Business logic
- ❏ Use: Mockito

```
import org.junit.jupiter.api.Test;
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;

import java.util.List;

class UserServiceTest {

    @Test
    void testGetAllUsers() {

        UserRepository mockRepo = mock(UserRepository.class);

        when(mockRepo.findAll()).thenReturn(List.of(new User()));

        UserService service = new UserService(mockRepo);

        List<User> users = service.getAllUsers();
    }
}
```

```
    assertEquals(1, users.size());
    verify(mockRepo).findAll();
}
}
```

□ 5. Controller Layer Testing (Web Layer)

- Focus: API endpoints
- Use: MockMvc

```
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

import java.util.List;

@WebMvcTest(UserController.class)
class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService service;

    @Test
    void testGetUsers() throws Exception {

        when(service.getAllUsers()).thenReturn(List.of(new User()));

        mockMvc.perform(get("/users"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.length()").value(1));
    }
}
```

❑ 6. Repository Layer Testing (Integration Test)

- ❑ Focus: Database interaction
- ❑ Use: H2 (in-memory DB)

```
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;

import static org.junit.jupiter.api.Assertions.*;

@DataJpaTest
class UserRepositoryTest {

    @Autowired
    private UserRepository repo;

    @Test
    void testSaveUser() {

        User user = new User();
        user.setName("Anurag");

        User saved = repo.save(user);

        assertNotNull(saved.getId());
        assertEquals("Anurag", saved.getName());
    }
}
```

❑ 7. Full Integration Test (End-to-End)

```
import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class UserApplicationTest {

    @Test
    void contextLoads() {
    }
}
```

❑ 8. Testing Annotations Cheat Sheet

Annotation	Purpose
@SpringBootTest	Full app test
@WebMvcTest	Controller test
@DataJpaTest	Repository test
@MockBean	Mock Spring bean
@Mock	Mockito mock
@InjectMocks	Inject mocks
@ExtendWith(MockitoExtension.class)	Enable Mockito

❑ 9. Best Practices (IMPORTANT)

- ✓ Test each layer separately
- ✓ Use mocks for Service & Controller
- ✓ Use H2 for Repository
- ✓ Avoid hitting real DB/API
- ✓ Keep tests fast & isolated

❑ 10. Real Company Approach (VERY IMPORTANT)

Layer	Strategy
Controller	Mock Service
Service	Mock Repository
Repository	Real DB (H2)
E2E	Full SpringBootTest

✓ Final Summary

- **Controller Test** → **MockMvc + @WebMvcTest**
- **Service Test** → **Mockito**

- **Repository Test** → **@DataJpaTest + H2**
 - **Integration** → **@SpringBootTest**
-

🔗 **If you want next:**

I can give you:

- ☐ Complete Spring Boot project (CRUD + tests)
- ☐ 50+ Interview questions (Spring Testing)
- ☐ Advanced topics (TestContainers, WireMock, REST Docs)

Just tell me ☐

Here's a deep, practical guide to Advanced Mockito covering:

- ✓ @Spy (partial mocking)
 - ✓ ArgumentCaptor (capture arguments)
 - ✓ doThrow, doAnswer, doReturn (advanced stubbing)
 - ✓ Real-world Java examples
-

☐ **1. Mockito Spy (Partial Mocking)**

☐ **What is Spy?**

- Real object + selective mocking
 - Calls real methods unless stubbed
-

🔗 **Example: Spy**

```
import org.junit.jupiter.api.Test;
import java.util.ArrayList;
import java.util.List;

import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;

class SpyTest {

    @Test
    void testSpy() {
```

```
List<String> realList = new ArrayList<>();

List<String> spyList = spy(realList);

spyList.add("A");
spyList.add("B");

// Real method call
assertEquals(2, spyList.size());

// Stub method
when(spyList.size()).thenReturn(100);

assertEquals(100, spyList.size());

// Verify interaction
verify(spyList).add("A");
}
```

🔗 Important Note

✗ Avoid this:

```
when(spy.method()).thenReturn(value); // may call real method
```

✓ Use this:

```
doReturn(value).when(spy).method();
```

❑ 2. ArgumentCaptor (Capture Arguments)

❑ Used to inspect values passed to a mock

🔗 Example: Email Service

```
class EmailService {
    void sendEmail(String to, String message) {
        // send email
    }
}
```

```
class NotificationService {

    private EmailService emailService;
```

```

NotificationService(EmailService emailService) {
    this.emailService = emailService;
}

void notifyUser() {
    emailService.sendEmail("user@test.com", "Welcome!");
}
}

```

❏ **Test using ArgumentCaptor**

```

import org.junit.jupiter.api.Test;
import org.mockito.ArgumentCaptor;

import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;

class ArgumentCaptorTest {

    @Test
    void testCaptor() {

        EmailService mockEmail = mock(EmailService.class);

        NotificationService service = new NotificationService(mockEmail);

        service.notifyUser();

        ArgumentCaptor<String> emailCaptor = ArgumentCaptor.forClass(String.class);
        ArgumentCaptor<String> msgCaptor = ArgumentCaptor.forClass(String.class);

        verify(mockEmail).sendEmail(emailCaptor.capture(), msgCaptor.capture());

        assertEquals("user@test.com", emailCaptor.getValue());
        assertEquals("Welcome!", msgCaptor.getValue());
    }
}

```

❏ **3. doThrow (Exception Testing)**

❏ Used for **void methods or forced exceptions**

❏ **Example**

```

class PaymentService {
    void processPayment() {
        // real logic
    }
}

```

```
}  
}
```

❏ Test

```
import org.junit.jupiter.api.Test;  
import static org.mockito.Mockito.*;  
  
class DoThrowTest {  
  
    @Test  
    void testException() {  
  
        PaymentService mockService = mock(PaymentService.class);  
  
        doThrow(new RuntimeException("Payment Failed"))  
            .when(mockService).processPayment();  
  
        try {  
            mockService.processPayment();  
        } catch (Exception e) {  
            assert e.getMessage().equals("Payment Failed");  
        }  
  
        verify(mockService).processPayment();  
    }  
}
```

❏ 4. doReturn vs when (IMPORTANT INTERVIEW)

Feature	when().thenReturn()	doReturn().when()
Calls real method?	✓ Yes (for spy)	✗ No
Safe for Spy	✗ No	✓ Yes
Syntax	Simple	Slightly verbose

❏ Example

```
List<String> spyList = spy(new ArrayList<>());  
  
// Wrong (may call real method)  
when(spyList.get(0)).thenReturn("Hello");  
  
// Correct  
doReturn("Hello").when(spyList).get(0);
```

❑ 5. doAnswer (Dynamic Behavior)

- ❑ Used when return depends on input
-

🔗 Example

```
import static org.mockito.Mockito.*;

class DoAnswerTest {

    @Test
    void testDoAnswer() {

        List<String> mockList = mock(List.class);

        doAnswer(invocation -> {
            String arg = invocation.getArgument(0);
            return "Hello " + arg;
        }).when(mockList).get(anyInt());

        String result = mockList.get(1);

        System.out.println(result); // Hello 1
    }
}
```

❑ 6. Real-Time Scenario (VERY IMPORTANT)

Scenario	Feature
Modify only some methods	Spy
Validate API request payload	ArgumentCaptor
Simulate failures (DB/API)	doThrow
Avoid real method execution	doReturn
Dynamic response (AI/API)	doAnswer

□ 7. Combined Example (Enterprise Style)

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock
    PaymentService paymentService;

    @InjectMocks
    OrderService orderService;

    @Test
    void testOrderFailure() {

        doThrow(new RuntimeException("Payment Failed"))
            .when(paymentService).processPayment();

        Exception ex = assertThrows(RuntimeException.class, () -> {
            orderService.placeOrder();
        });

        assertEquals("Payment Failed", ex.getMessage());
    }
}
```

□ 8. Interview Summary (Must Remember)

Concept	Key Point
Spy	Partial mocking
ArgumentCaptor	Capture method arguments
doThrow	Test exceptions
doReturn	Safe stubbing for spy
doAnswer	Custom logic

✓ Final Understanding

- **Spy** = Real + Mock
- **Captor** = Inspect arguments
- **doThrow** = Force exception
- **doReturn** = Safe stub
- **doAnswer** = Dynamic logic

🔗 Want next level?

I can give you:

- ☐ 50+ Mockito Interview Questions (FAANG level)
- ☐ Spring Boot + Mockito real project (end-to-end)
- ☐ TestContainers + WireMock (industry-level testing)

Just tell me ☐